



Tema 17: Deep Learning II

Contenidos

- Arquitectura *encoder – decoder*.
- Autoencoders variacionales.
- Modelos de difusión.
- Redes generativas adversarias.

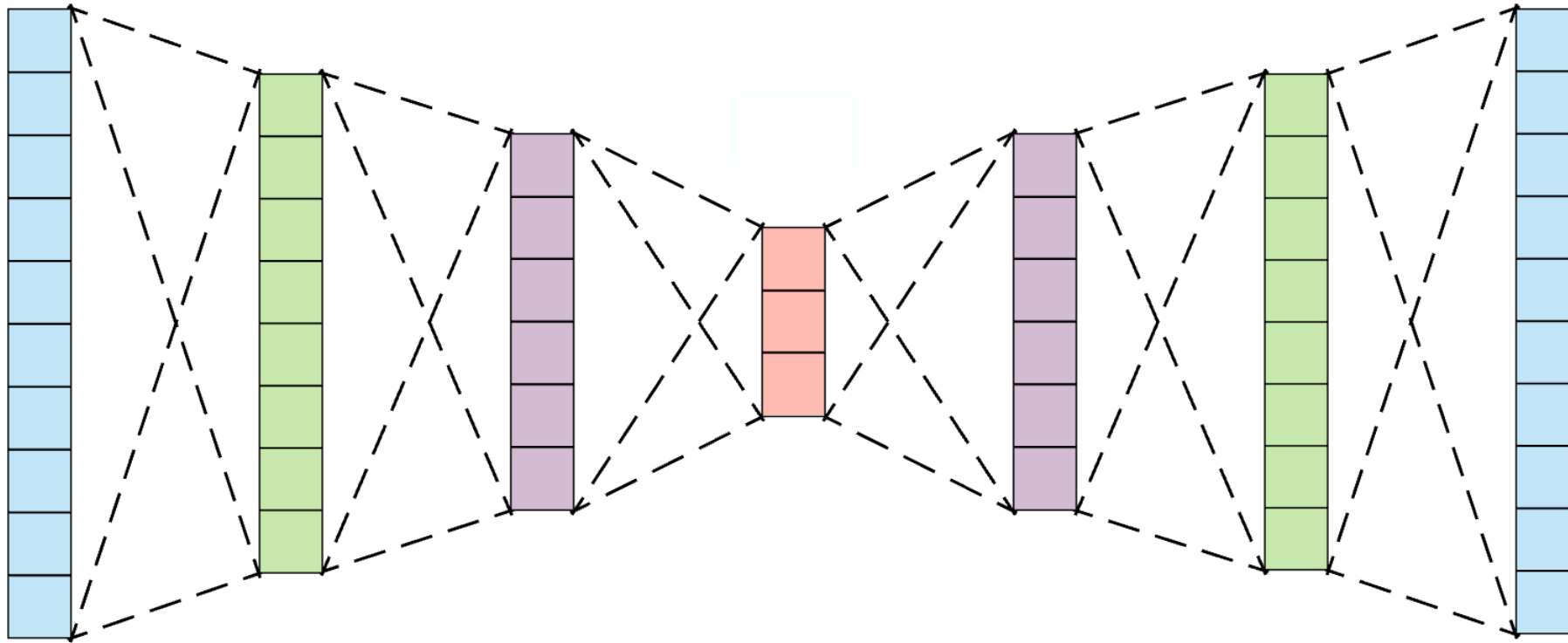


Arquitectura encoder-decoder



Estructura codificador – decodificador con un cuello de botella.

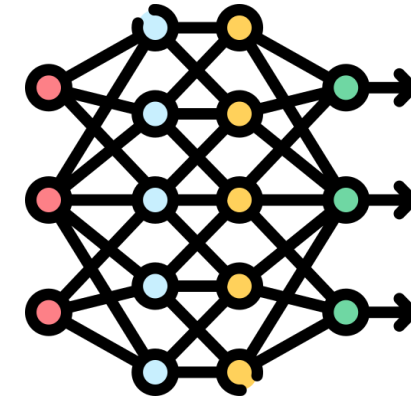
Es una red neuronal cuyo objetivo es que los *outputs* sean iguales a los *inputs*.



¿Qué hacen los *autoencoders*?

Un *autoencoder* comprime la información.

- Forma un espacio latente.
- Aprende a seleccionar la información importante para representar los datos. Aquella que aporta valor.
- Permite hacer una reducción de dimensionalidad muy buena.



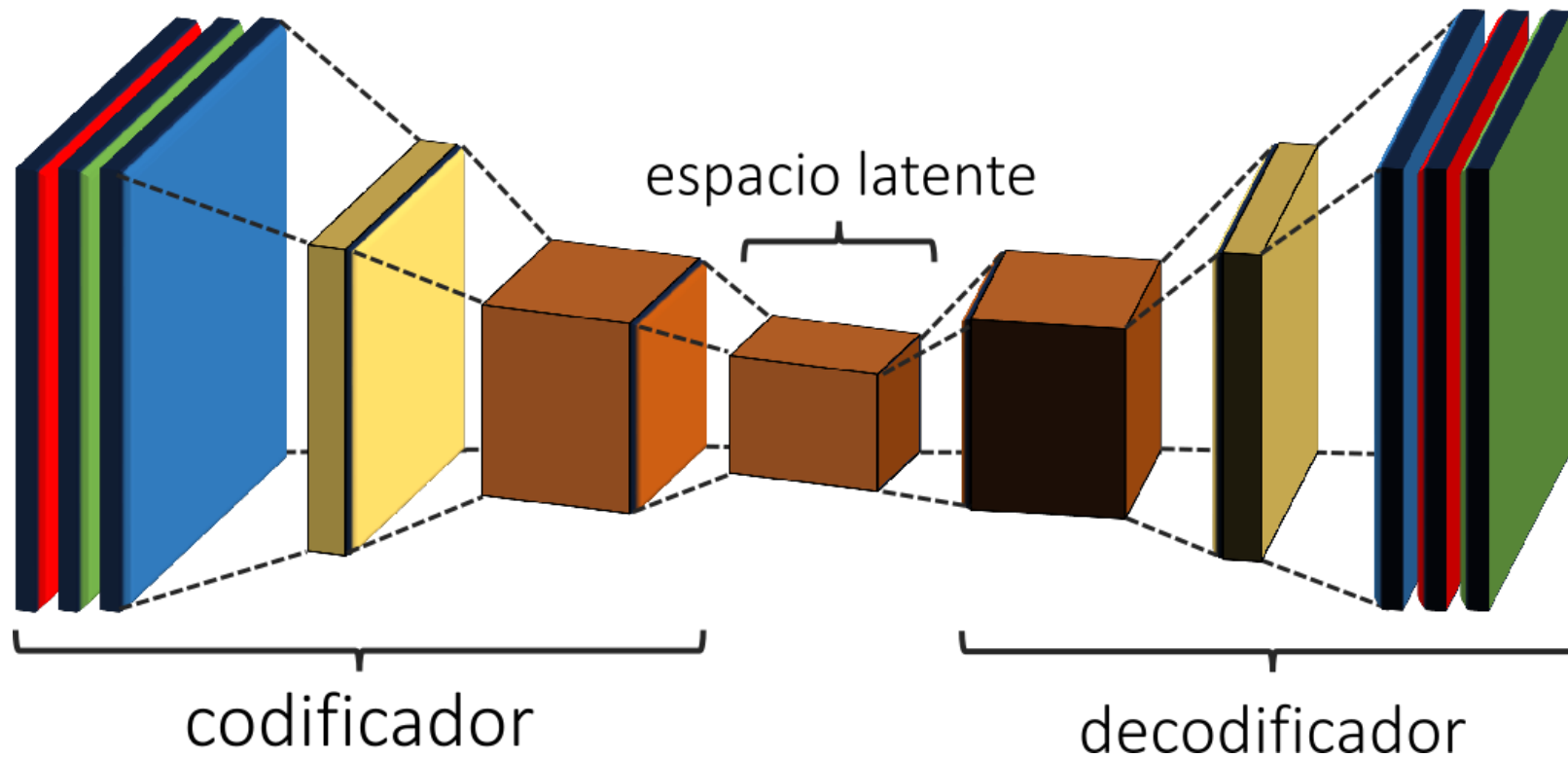
Diversas aplicaciones:

- Detección de anomalías.
- Embeddings multimodales – unificación de tipos de datos.
- Modelos generativos.
- Simplificación de problemas de manera no supervisada.



Estructura de un *autoencoder*

Cuando tiene varias capas, se llama *stacked autoencoder*.



Tenemos un conjunto de datos de entrada x_i .

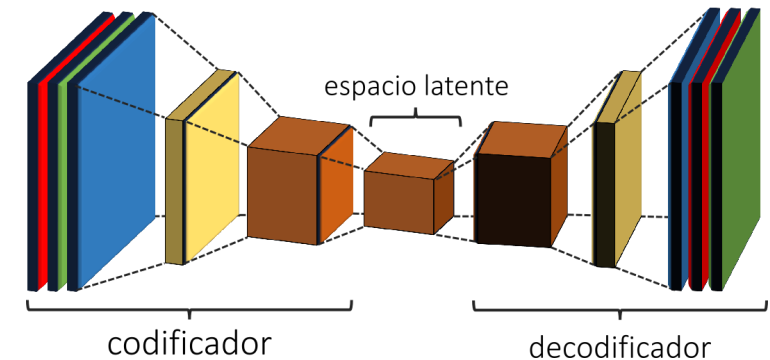
El *autoencoder* incluye:

- Codificador: $f(x) = h$. La dimensión de h es normalmente menor que la de x , y se llama representación latente. Es una red neuronal con parámetros θ_c .
- Decodificador: $g(h) = \hat{x}$. La dimensión de $\hat{x}$ es la misma que la de x . Es una red con parámetros θ_d .

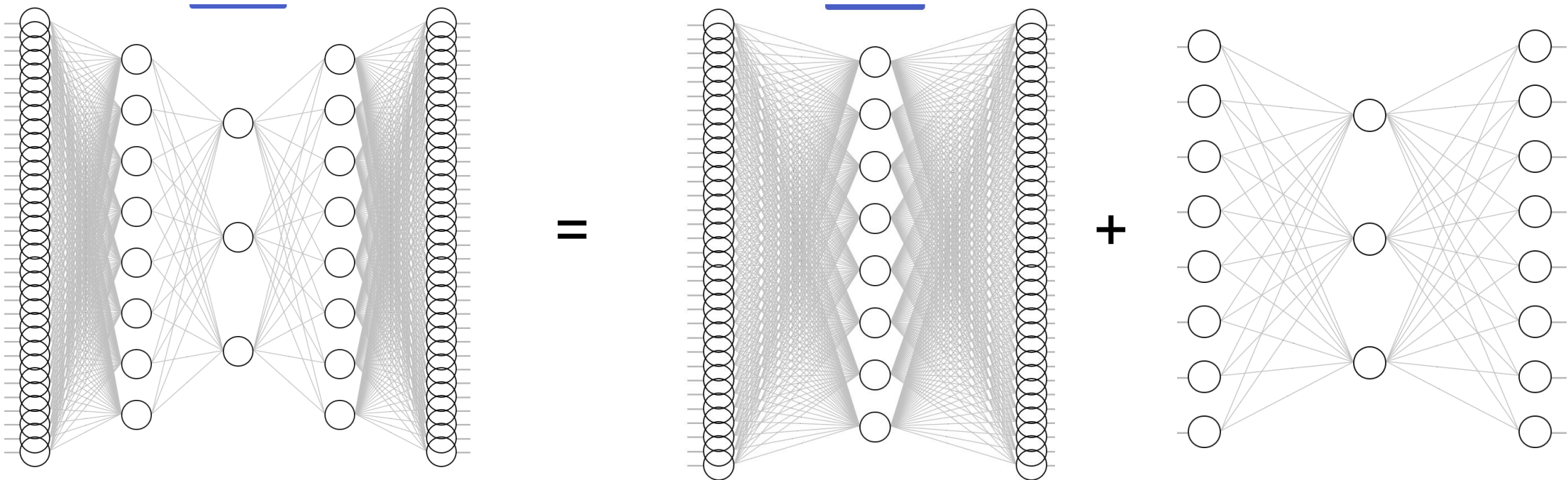
En general, podemos construir una función de pérdida:

$$J(\theta_c, \theta_d; x, \hat{x}) = \frac{1}{N} \sum_i^N |x_i - \hat{x}_i|^2$$

Utilizamos esta función de pérdida para entrenar el modelo.



Una estrategia de entrenamiento de *stacked autoencoders* es hacerlo por capas:

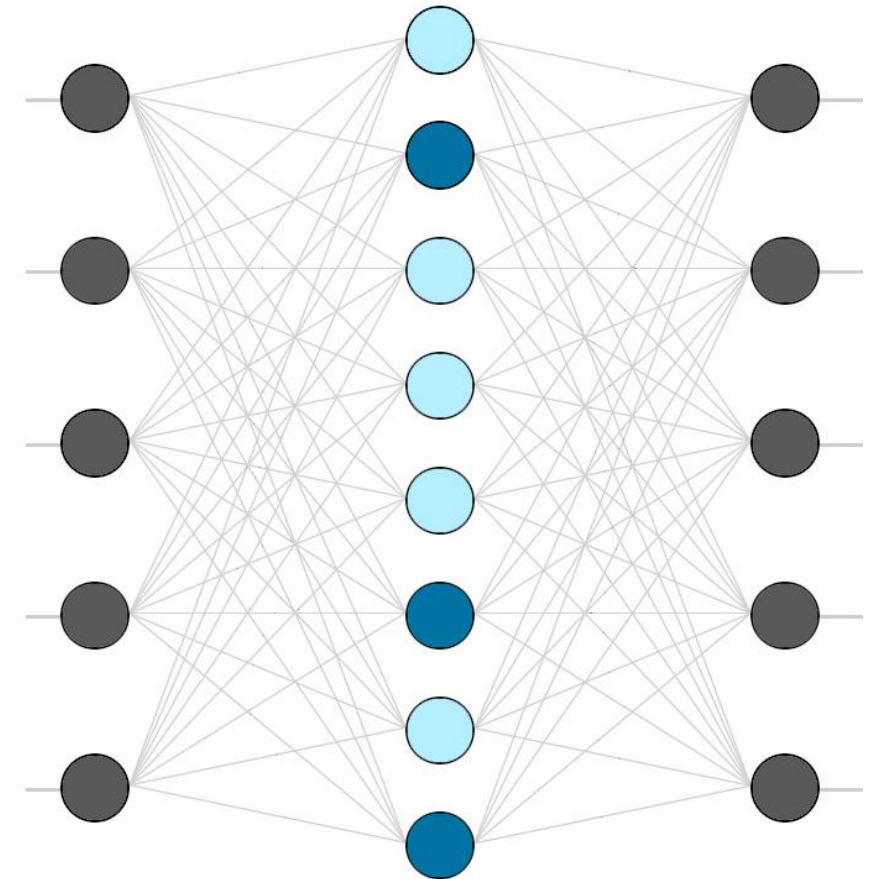


Son *autoencoders* que se entrenan para que solo algunas neuronas estén activas. En general, tienen las siguientes características:

- Solo una capa oculta (aunque puede haber *stacked* SAE)
- Más neuronas en la capa oculta que en las de input/output
- Aprende a seleccionar la información importante para representar los datos. Aquella que aporta valor.

Para conseguir esto, se utiliza una penalización a la función de pérdida del tipo *KL-divergence*:

$$J(\theta_c, \theta_d; x, \hat{x}) = \frac{1}{N} \sum_i^N |x_i - \hat{x}_i|^2 + \beta \sum_j^{d_h} KL(\rho_j || \hat{\rho}_j)$$



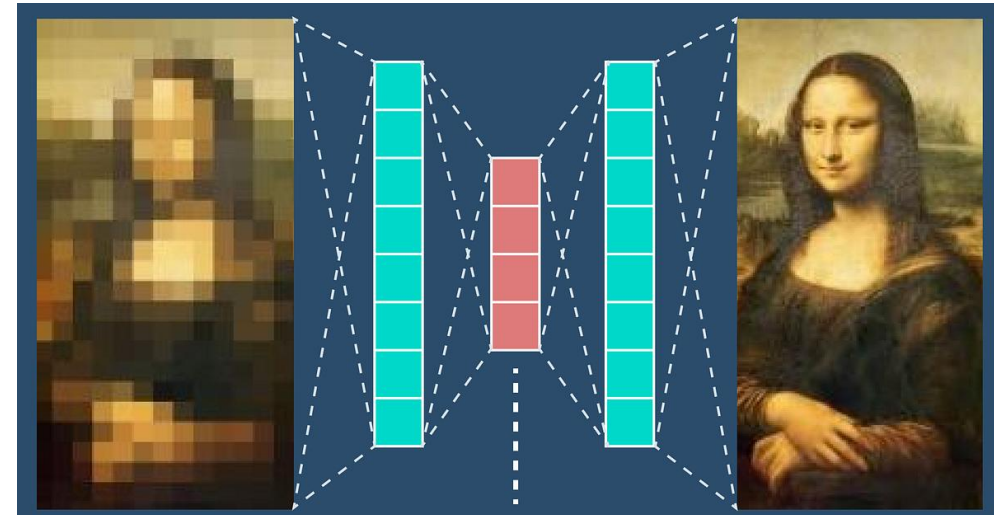
Los *denoising autoencoders* aprenden a reconstruir los *inputs* sin ruido. Para ello:

- Partimos de inputs “limpios” x .
- Añadimos ruido generado para obtener $\tilde{x}$. Por ejemplo, $\tilde{x} = x + r$, donde $r \sim N(0, \sigma^2)$.
- Entrenamos el autoencoder con $\tilde{x}$ como *input* y x como *output*.

La función de pérdida sería:

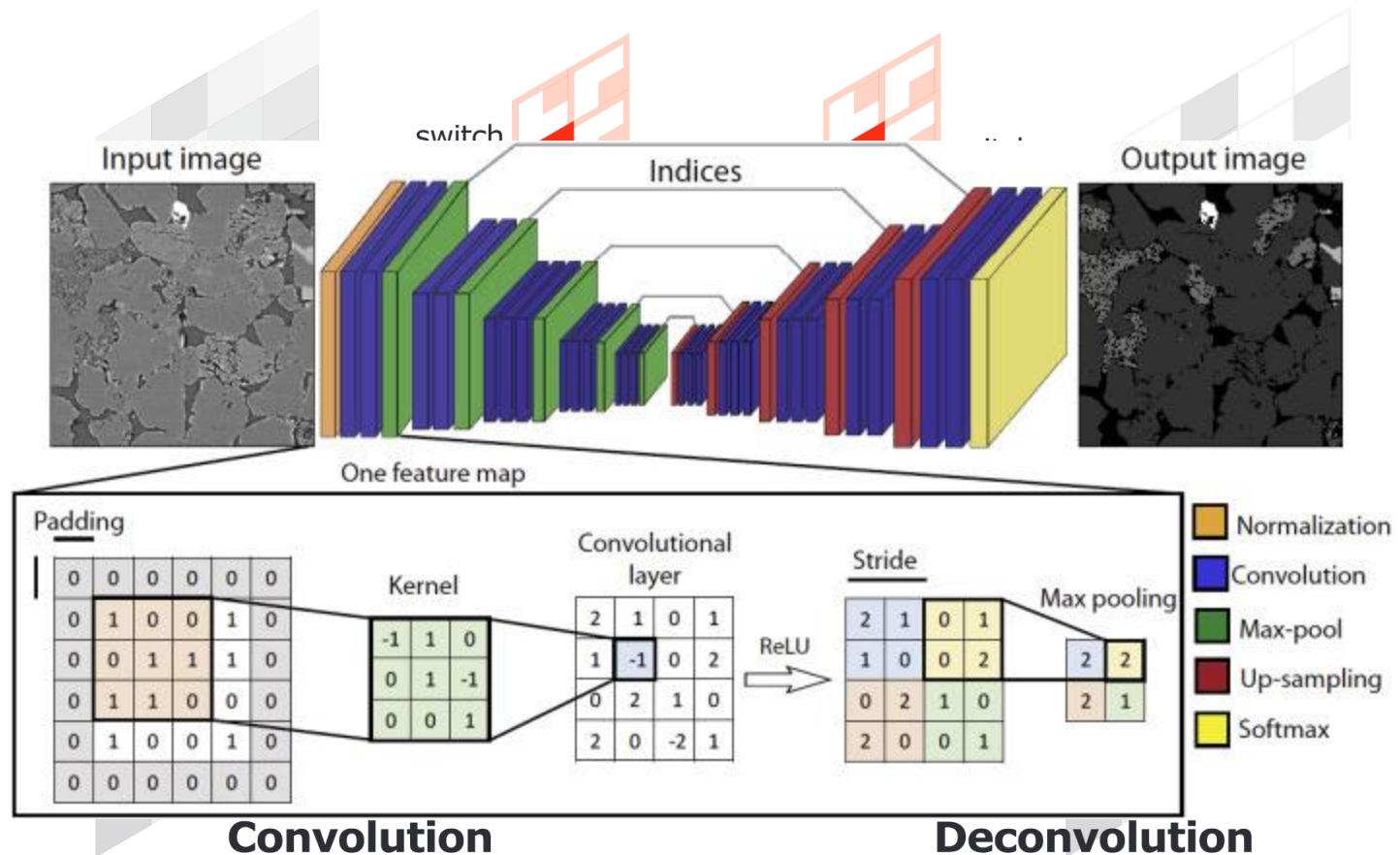
$$J(\theta_c, \theta_d; x, \hat{x}) = \frac{1}{N} \sum_i^N |x_i - \hat{x}_i|^2 = \frac{1}{N} \sum_i^N |x_i - g_{\theta_d}(f_{\theta_c}(\tilde{x}_i))|^2$$

Son muy útiles cuando tenemos datos ruidosos.



Los *autoencoders* convolucionales tienen:

- Codificador: tiene la estructura habitual de una CNN de extracción de características.
- Decodificador: contienen capas deconvolucionales, que realizan la operación inversa a la convolución.



Un *autoencoder* normal ve una imagen como una lista de píxeles, uno convolucional la ve como una imagen con patrones y características.

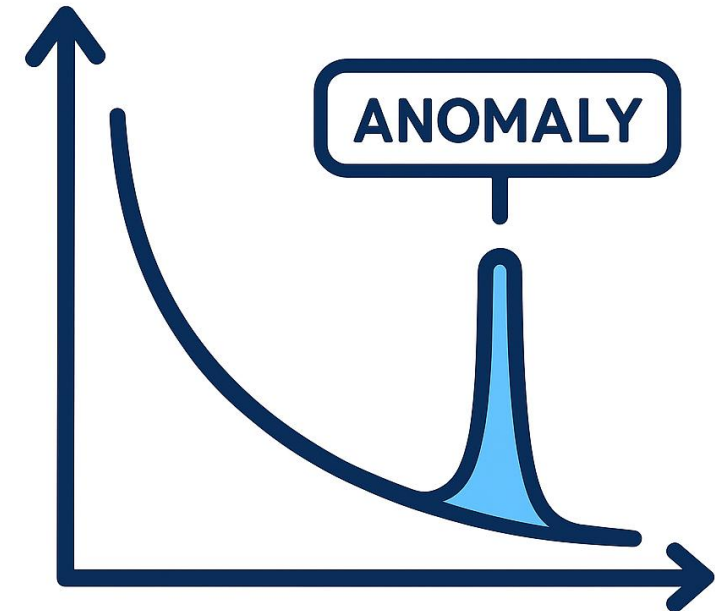
Podemos usar *autoencoders* para clasificar / clusterizar datos no etiquetados. Útil para obtener diferentes clases incluso cuando no conocemos las categorías.

1. Se entrena el *autoencoder*.
2. Se obtienen las representaciones latentes del *dataset*.
3. Se utiliza un algoritmo de *clustering* sobre los vectores latentes.



Gracias a la capacidad de “aprender lo esencial”, se pueden utilizar para detectar anomalías:

1. Entrenamos con datos normales (no anómalos).
2. Evaluamos el AE sobre más datos normales → baja pérdida de reconstrucción.
3. Evaluamos sobre un dato anómalo → alta pérdida debido a que el AE no se entrenó con datos anómalos.

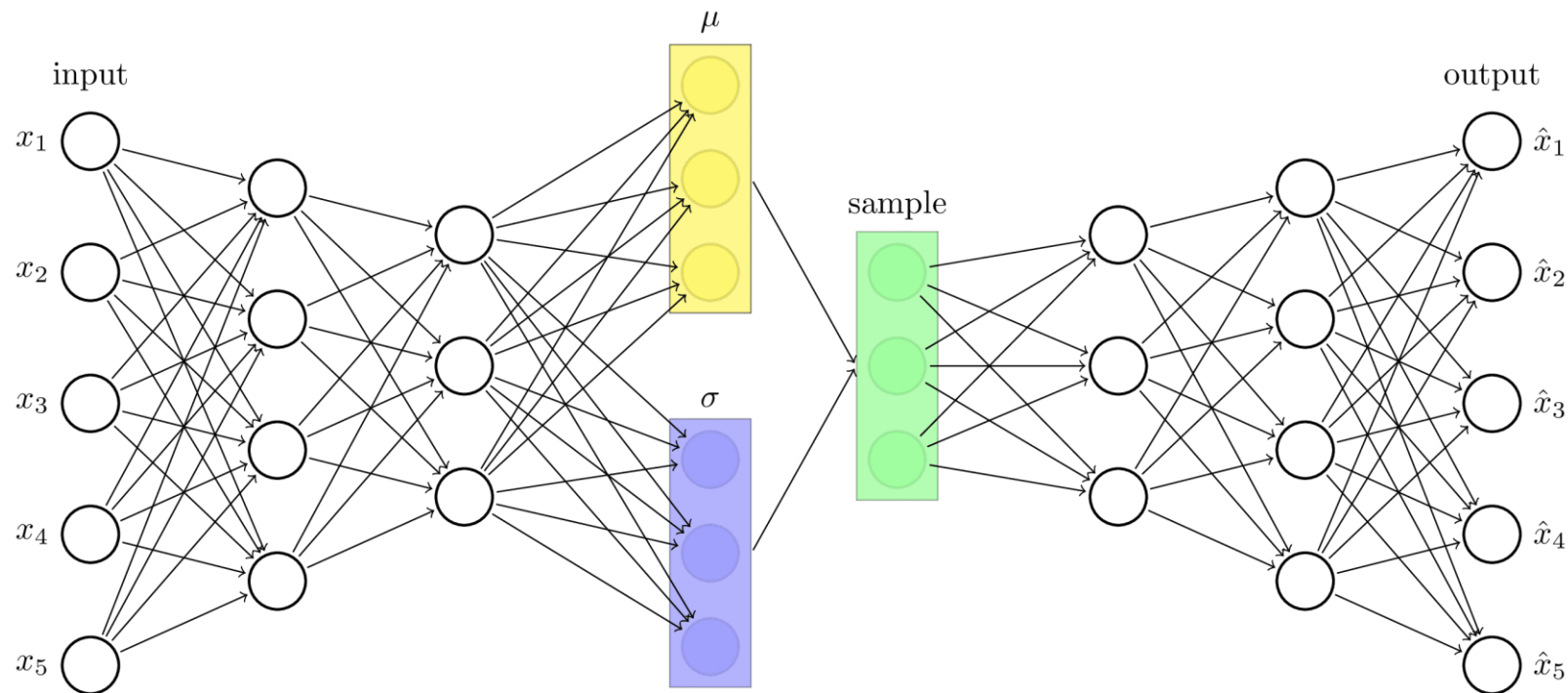


Autoencoders variacionales



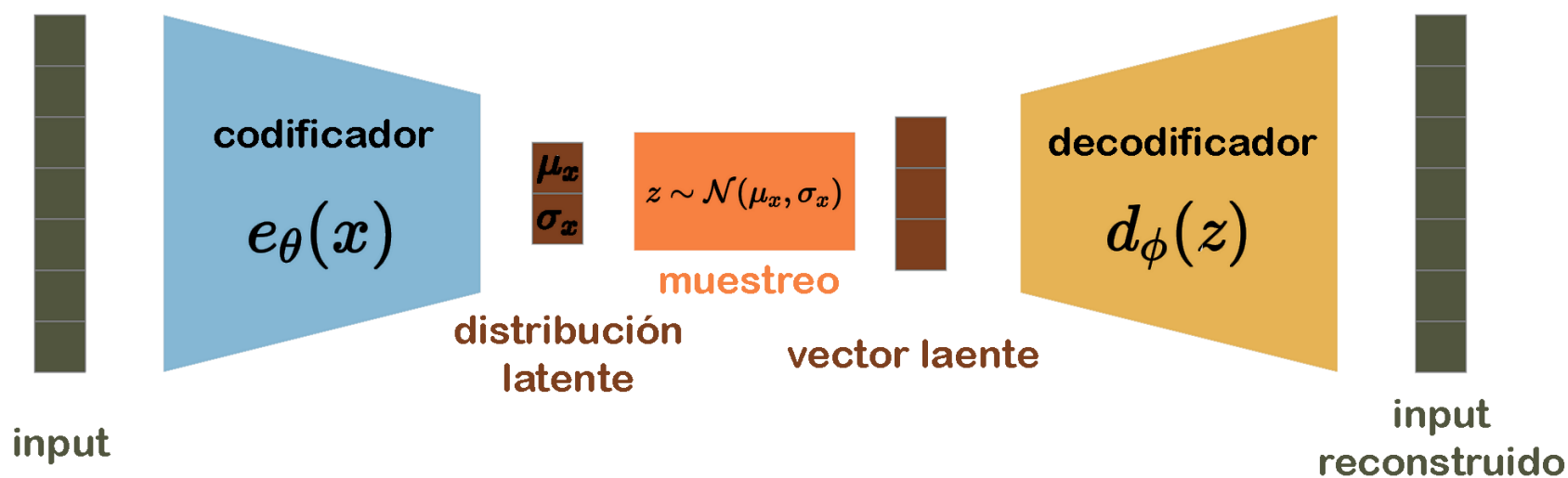
Los VAE aprenden las **distribuciones de probabilidad** de los datos de entrada.

Una vez aprendidas, se pueden utilizar para **generar nuevos datos realistas**: se pueden usar como modelos generativos.



La representación latente no se hace a través de características h , sino que se asume una variable del espacio latente z con una distribución, típicamente normal.

- Como función de coste se utiliza la evidencia de $p_{\theta_d}(x|z)$ usando, por ejemplo, ELBO (*evidence lower bound*).
- Generamos datos muestreando el espacio latente.



Podemos crear un espacio latente “ordenado”

Por ejemplo, podemos entrenar un VAE con MNIST y luego explorar el espacio latente entre dos imágenes concretas.

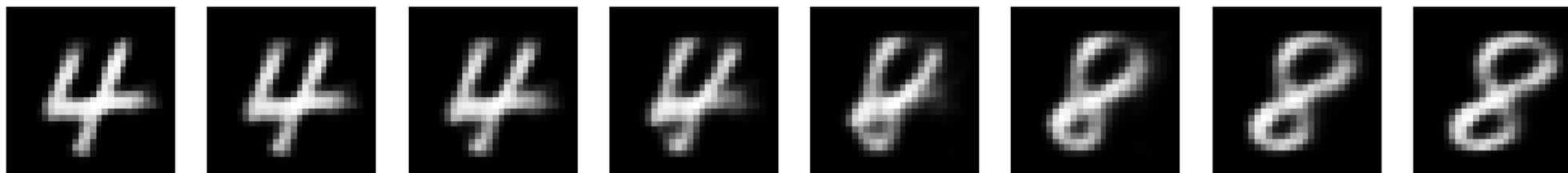
- Por ejemplo, entre un 0 y un 7:

Interpolación entre dígito 0 y dígito 7



- 0 entre un 4 y un 8:

Interpolación entre dígito 4 y dígito 8



Aplicación: generación de imágenes



Modelos de difusión



Modelos de generación de imágenes

- Redes generativas adversariales (GANs): primer modelo de generación de imágenes realistas (~2014). Generación robusta, dominio específico, muy rápido. Ejemplo: *styleGAN*, *ERS GAN*...
- Modelos de difusión: auge en ~2020. Modelos totalmente generales. Ampliamente acoplados con *inputs* textuales – se pueden usar con *prompts* para generación específica. Ejemplos: *stable diffusion*, *dall·e*, *midjourney*...
- Autorregresivos (~2023): tecnología más avanzada. Forma parte de modelos multimodales. Ejemplos: *GPT4o*, *Nano Banana (Gemini)*...



Más adelante veremos el funcionamiento de las GANs. Pero ahora, vamos a ver brevemente cómo funcionan los modelos de difusión.

- Evolución a partir de *autoencoders*.
- Necesitamos repasar algunos conceptos:
 1. *Embeddings* (de texto, por ejemplo).
 2. Conexiones residuales.
 3. Arquitectura U-net.
 4. Generación y reducción de ruido (*denoising*).



DALL·E 3



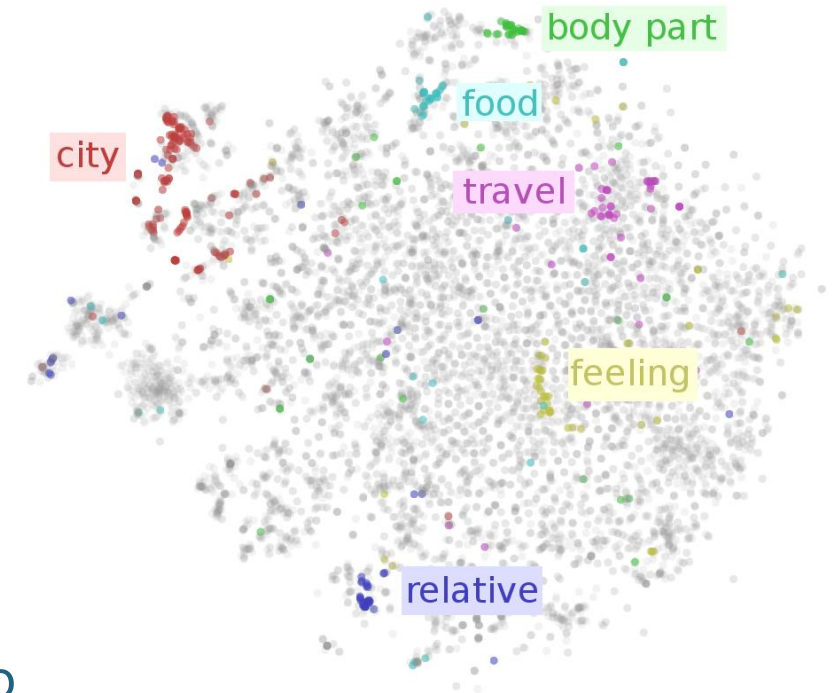
Midjourney



Codificación de elementos en forma de vector. Lo más conocido son los *Word Embeddings*, pero se puede hacer *embeddings* de cualquier cosa.

Forman un espacio vectorial. Se pueden hacer cálculos con los vectores: distancias, sumas, *clustering*, búsquedas...

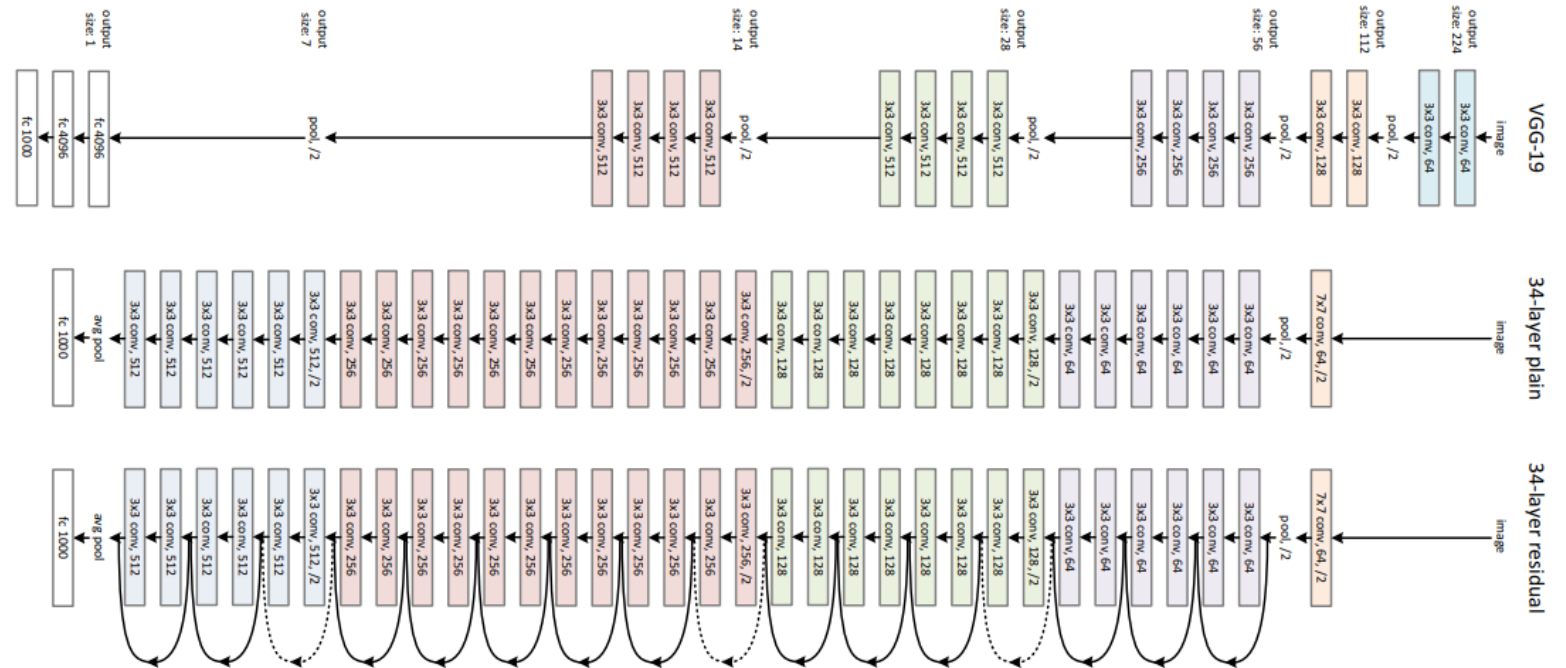
Los *embeddings* se usan para vectorizar cualquier cosa: palabras o caracteres, fragmentos de imágenes, secuencias de audio...



Conexiones residuales

También llamadas “*skip connections*”.

Aparecen en las ResNET.

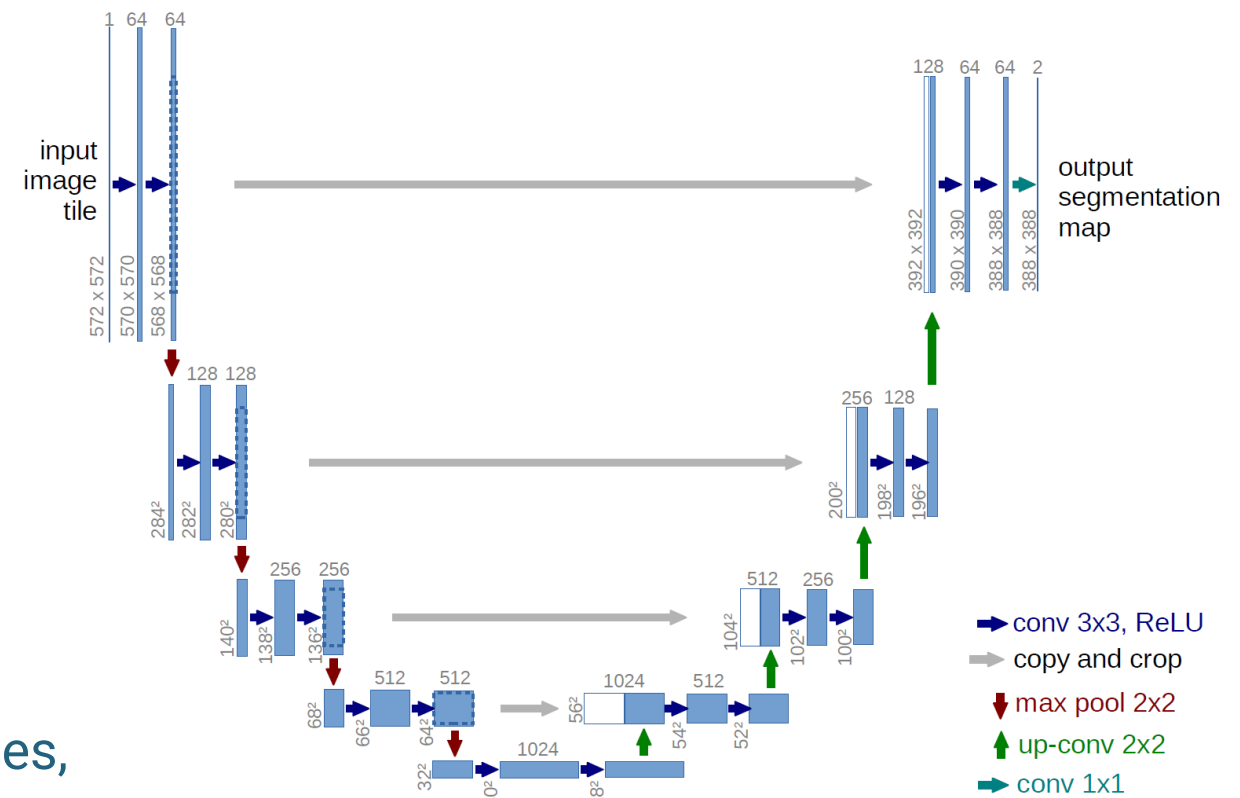


Son muy usadas en redes neuronales complejas, ya que ayudan mucho a que la red aprenda y funcione correctamente.



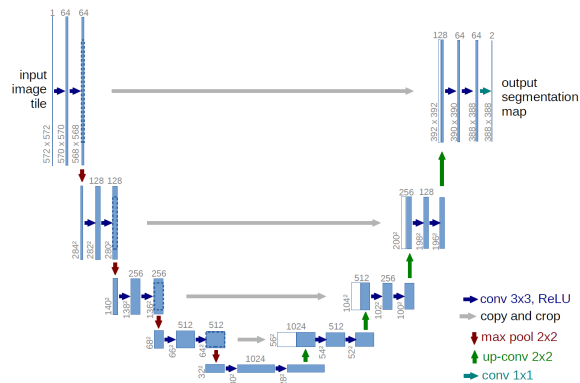
- Inicialmente abordaban el problema de segmentación de imágenes médicas.
- Se utilizan habitualmente para procesamiento de imágenes y, como vamos a ver, en modelos de difusión.

Autoencoder convolucional + conexiones residuales, llamadas en el paper original “copy and crop”, que se aplican a las imágenes/filtros en capas concretas.



Combinación de:

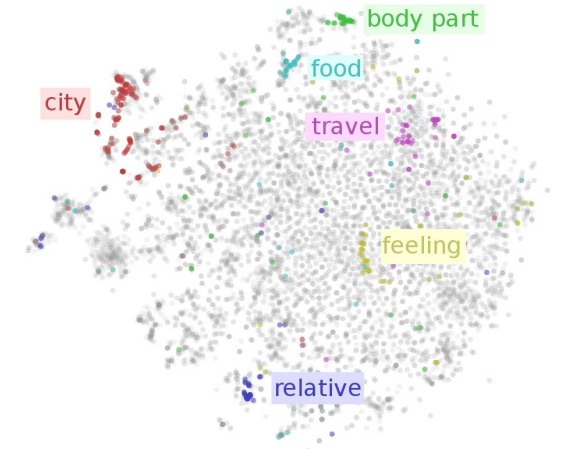
- U-net (que a su vez combina *autoencoder* convolucional y conexiones residuales).
- La idea de reducción de ruido de los *denoising autoencoders*.
- *Embeddings* para añadir información temporal/espacial + contexto.



+

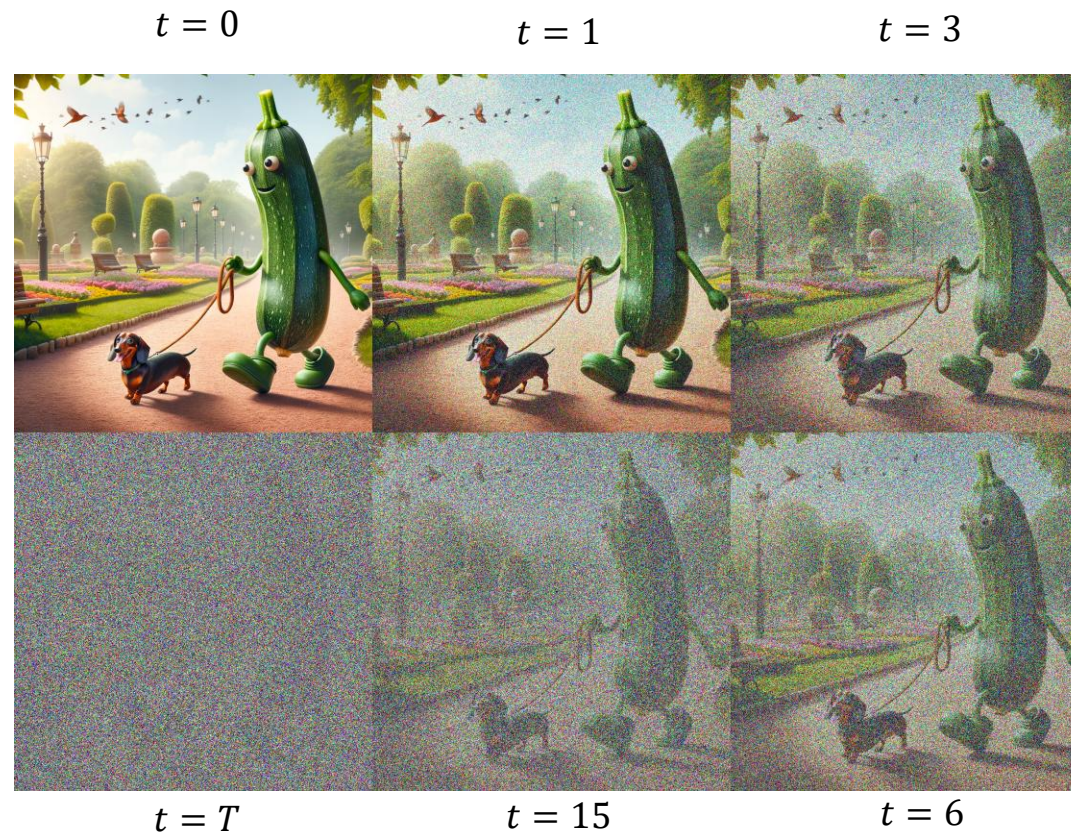


+

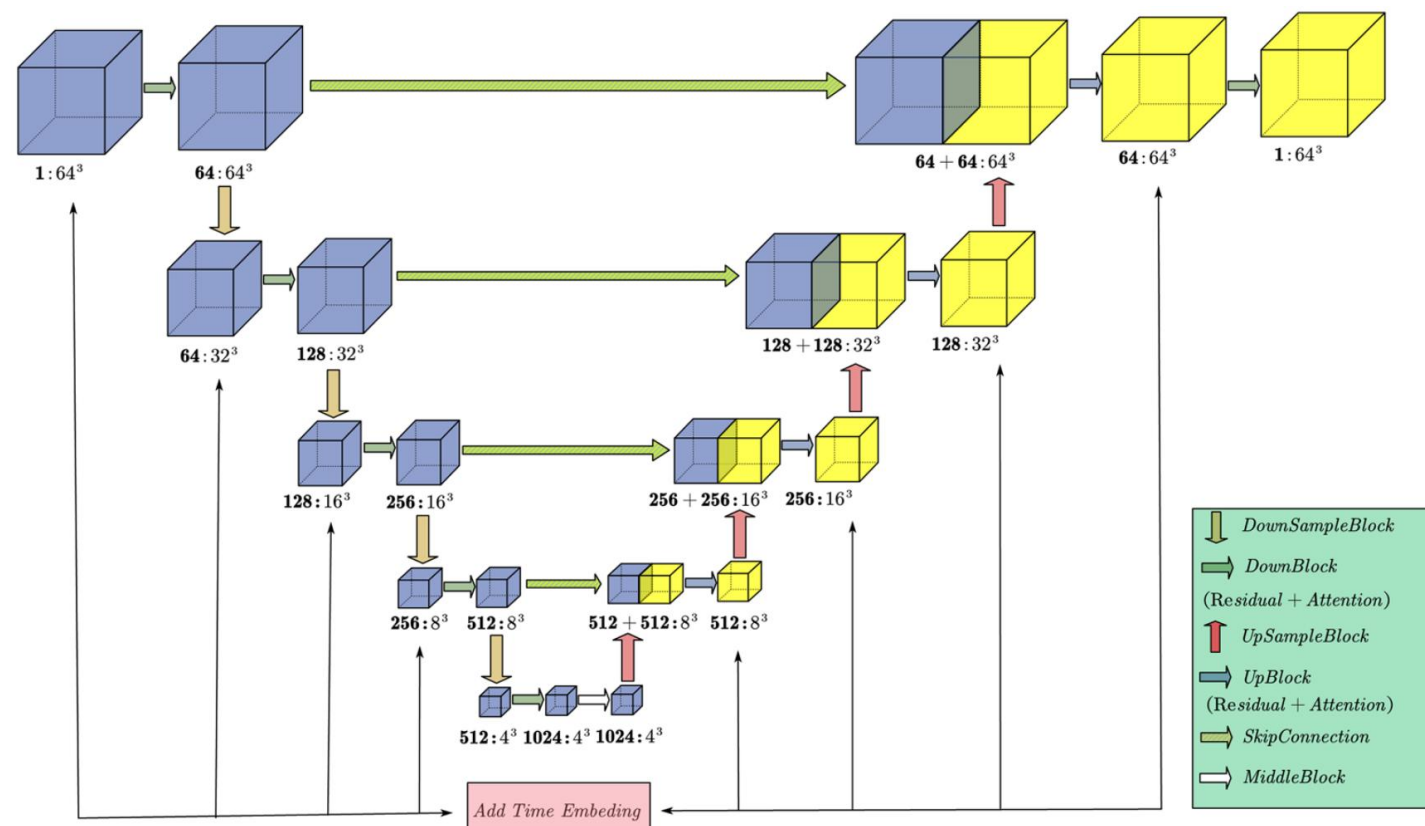


La idea clave: añadimos ruido a un elemento y entrenamos una red de tipo U-net para que aprenda a predecir ese ruido añadido.

- N pasos temporales, de $t = 0$ (*input* sin ruido) a $t = T$, 100% ruido.
- Cada paso t , se añade un poco de ruido. Se entrena al modelo para que lo prediga.
- Contexto en forma de *embedding* para condicionar la predicción del ruido.
- Generación: se parte de **ruido aleatorio** y se genera un nuevo elemento usando el proceso inverso: de $t = T$ a $t = 0$.



- El proceso de añadir/predecir ruido se realizar según un *scheduler*.
- Durante el proceso, se añade información condicional (categorías, *prompts* de texto, etiquetas...).
- Ruido aleatorio + información / *prompt* / etiquetas para condicionar la predicción de ruido.



Redes generativas adversariales



Se trata de un modelo que combina redes neuronales con diferentes funciones para conseguir buenos resultados en la generación de datos (como imágenes).



2014



2015



2016



2017

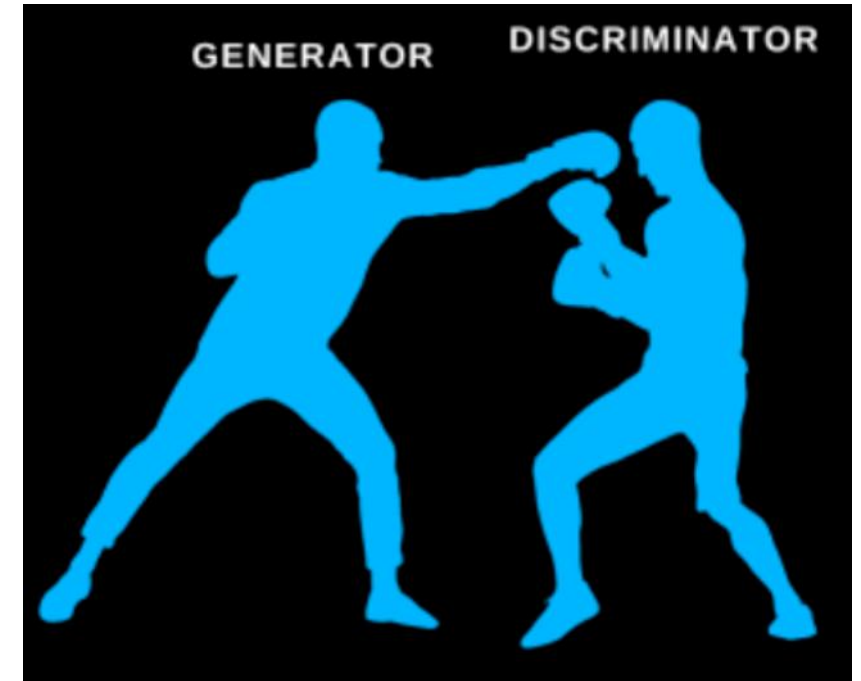


Actualidad



Las GANs constan de dos partes:

- Generador: una red similar al decodificador de un VAE.
- Discriminador: parecido a una CNN usada en clasificación binaria.

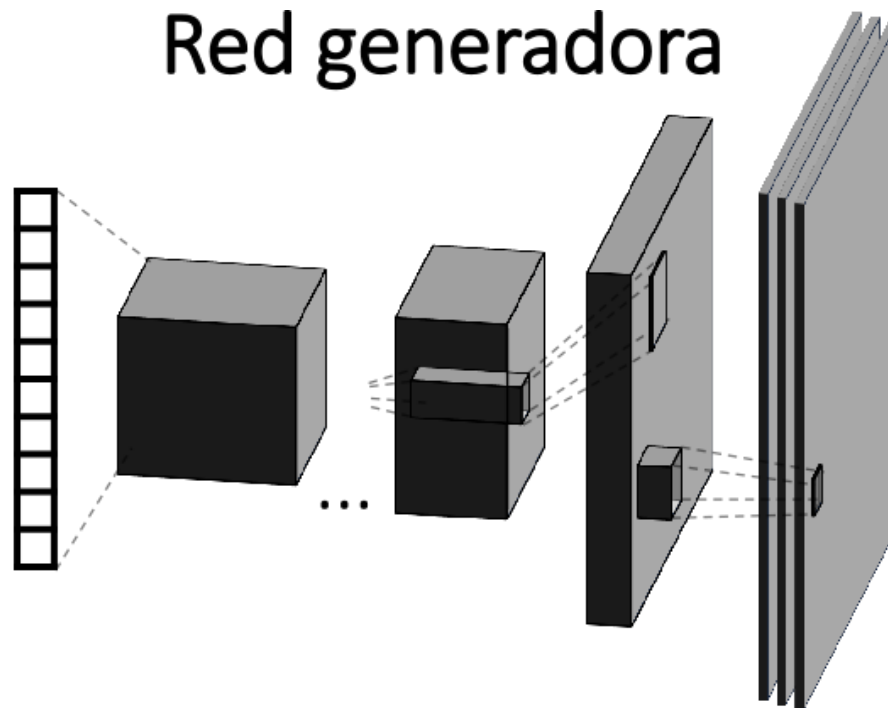


Las GANs combinan estas dos redes neuronales, que compiten para ir mejorando progresivamente, usando un *dataset* de entrenamiento no etiquetado (es decir, es aprendizaje no supervisado).



La red generadora G es similar al *decoder* de un *autoencoder* convolucional:

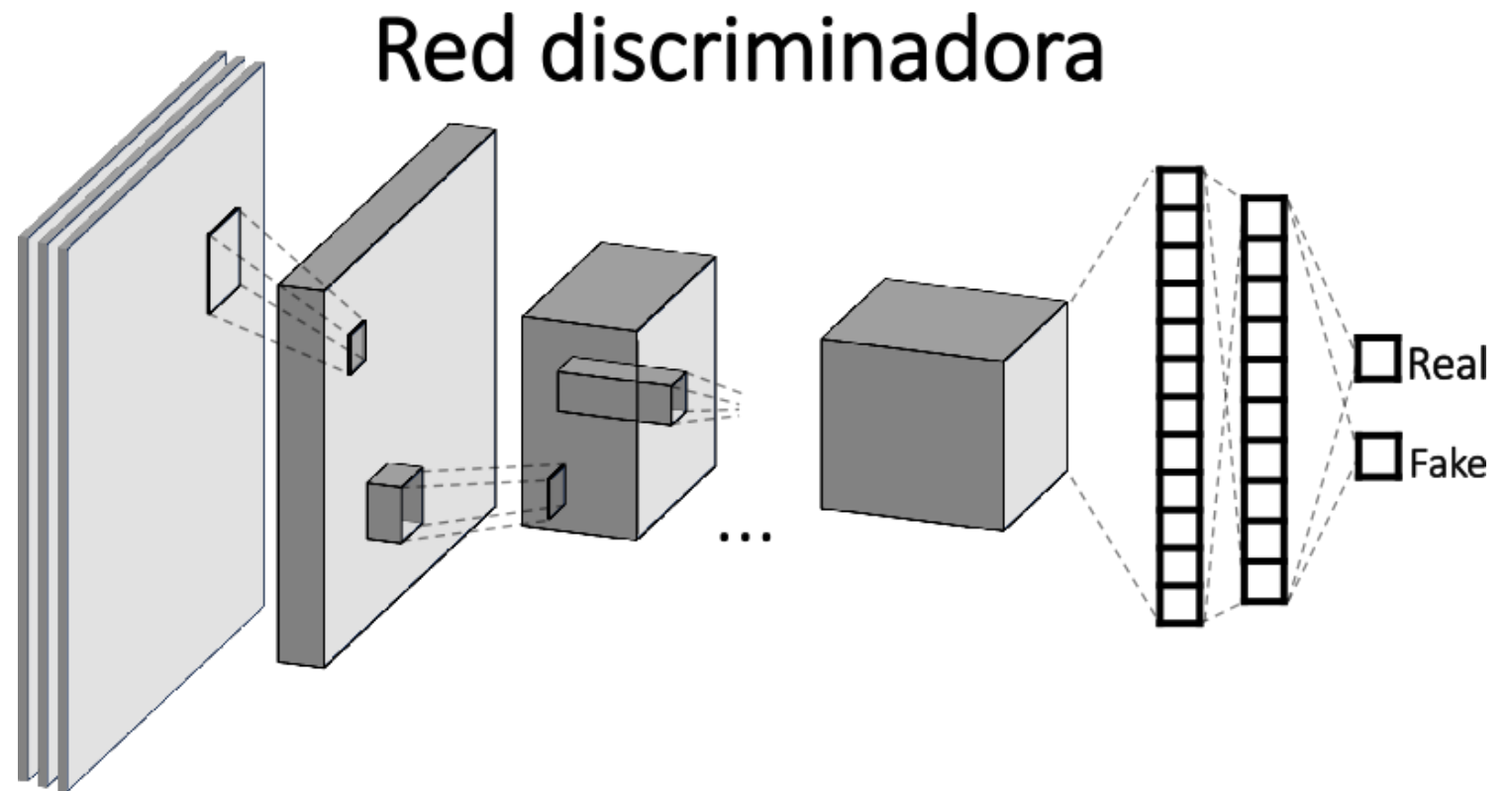
- Parte de un vector de ruido.
- Usa bloques de-convolucionales y *unpooling* hasta obtener una imagen con 3 canales.



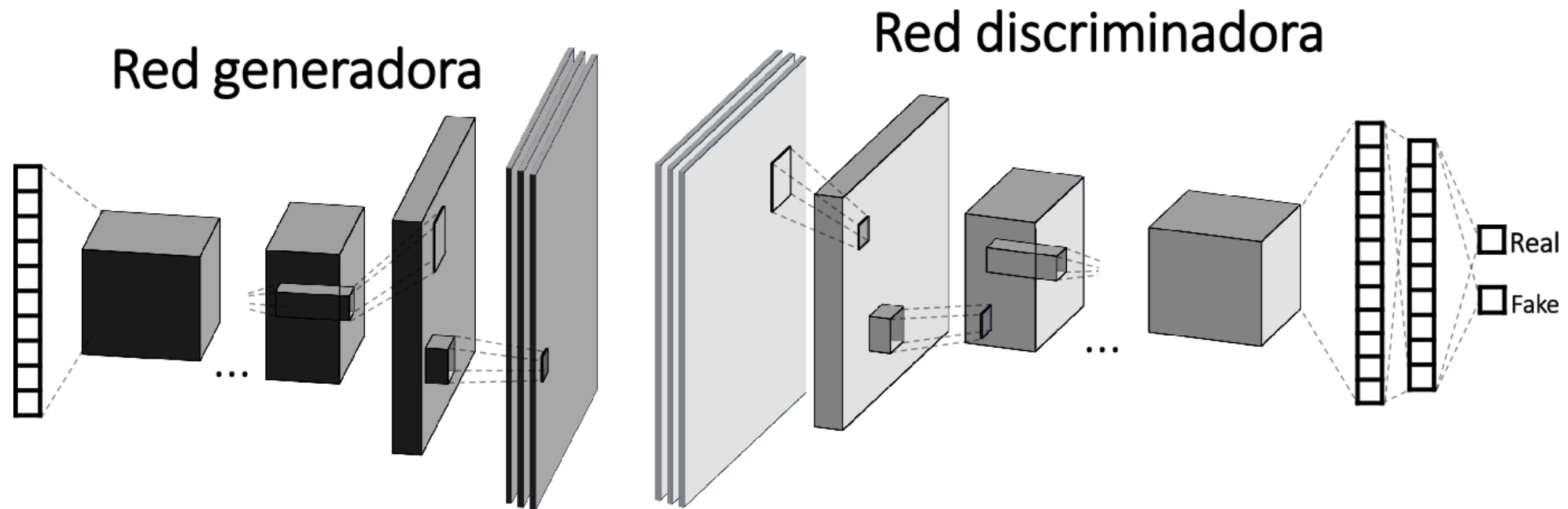
La diferencia fundamental es que la red generadora no maximiza una probabilidad, sino que se entrena a partir de la salida del discriminador.

La red discriminadora en una GAN es similar a una CNN + MLP clasificador binario con dos categorías finales: *real* y *fake*.

Su objetivo es identificar como *fake* las imágenes que produce el Generador, y como real imágenes del *dataset* de entrenamiento.

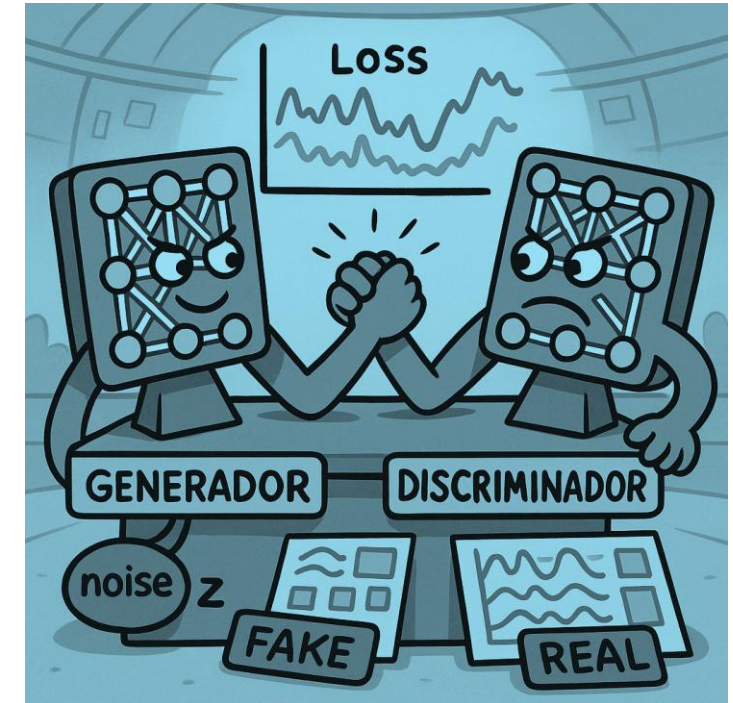


- Generador: parte de un vector de ruido y termina con un *output* de dimensión (N_c, W, H) .
- Discriminador: toma como *input* una imagen de dimensión (N_c, W, H) y devuelve una probabilidad de ser clasificada como *real* o *fake*.



1. Generación de imágenes.
 - a. Generamos un vector de ruido aleatorio $z \sim p(z)$.
 - b. Lo usamos como *input* del generador para obtener una imagen $G(z)$.
 - c. Repetimos para obtener un lote de datos generados B_{gen} .
2. Discriminador.
 - a. Leemos un lote de datos de imágenes reales B_{real} .
 - b. Obtenemos probabilidades de clasificación para B_{gen} y B_{real} .
 - c. Actualizamos D de acuerdo a estas probabilidades: maximizamos $\log [D(x)]$ para datos reales y $(1 - \log [D(G(z))])$ para datos generados.
3. Actualizamos el generador. Usando como función de pérdida:

$$\log(1 - D(G(B_{gen}))).$$



- Generación de imágenes
 - styleGAN [arXiv:1812.04948](https://arxiv.org/abs/1812.04948)
- Transferencia de estilo y dominio
 - pix2pix [arXiv:1611.07004](https://arxiv.org/abs/1611.07004)
 - CycleGAN [arXiv:1703.10593](https://arxiv.org/abs/1703.10593)
- Mejora de resolución
 - ERSGAN [arXiv:1809.00219](https://arxiv.org/abs/1809.00219)
 - Real-ERSGAN [arXiv:2107.10833](https://arxiv.org/abs/2107.10833)
- Otros formatos:
 - Audio – WaveGAN (github.com/chrisdonahue/wavegan)
 - Datos tabulares – CTGAN ([Modeling tabular data using conditional GAN](https://arxiv.org/abs/1907.07931))
 - Moléculas – MoleGAN ([arXiv:1805.11973](https://arxiv.org/abs/1805.11973))
 - Vídeo – StyleGAN-V (github.com/universome/stylegan-v)



Entrenando una GAN





EBIS Business
Techschool

www.ebiseducation.com

